Project Report

AI&ML

Stock Market Analyzer

Parth Agarwal

25BCE10497

Abstract

The AI ML Stock Market Analyzer is a web-based application developed to analyze stock market data using technical indicators, risk metrics, and machine learning prediction. The system downloads historical stock data, calculates indicators such as RSI, MACD, and Moving Averages, evaluates risk using volatility and Sharpe ratio, and predicts the next day closing price using a Linear Regression model. The application is built using Python, Streamlit, scikit-learn, and Plotly, providing an interactive dashboard for users to visualize stock trends and analysis. The project demonstrates the integration of financial analysis, data science, and machine learning into a single intelligent stock analysis system.

1. Problem Statement

Stock market analysis involves understanding price trends, technical indicators, and risk factors, which can be complex for beginners and retail investors. Many platforms provide stock

data but do not combine technical analysis, risk metrics, and machine learning prediction into one simple system.

This project aims to develop an AI-based stock market analyzer that can automatically download stock data, calculate technical indicators, analyze risk, visualize trends, and predict the next day closing price using machine learning. The system provides a user-friendly web interface where users can input a stock ticker and date range to perform complete stock analysis.

2. Objectives

- Develop a stock market analysis system using Python and Machine Learning
- Implement technical indicators such as RSI, MACD, and Moving Averages
- Calculate financial risk metrics such as volatility and Sharpe ratio
- Build a machine learning model for next day stock price prediction
- Create a web-based dashboard using Streamlit for visualization and interaction

- Provide downloadable analyzed data in CSV format
- Visualize stock price trends and indicators using interactive charts
- Allow users to select custom date ranges
- Evaluate prediction accuracy using MAE and R^2 score

3. Functional Requirements

3.1 Major Functional Modules

Module 1: Stock Data Collection

Input: Stock ticker symbol and date range
Output: Historical stock data

Functionality:

- Download stock data using Yahoo Finance API
- Clean and preprocess data
- Handle missing values
- Store data in a structured DataFrame

Module 2: Technical Indicator Calculation

Input: Historical stock prices

Output: RSI, MACD, Moving Averages, Volatility

Functionality:

- Calculate Relative Strength Index (RSI)
- Calculate Moving Average Convergence Divergence (MACD)
- Calculate moving averages (10, 20, 50 days)
- Calculate rolling volatility
- Generate lag features for machine learning model

Module 3: Risk Analysis Module

Input: Closing prices

Output: Risk metrics

Functionality:

- Calculate average daily return
- Calculate daily and annual volatility
- Calculate expected annual return
- Calculate Sharpe ratio
- Display results in tabular format

Module 4: Machine Learning Prediction Module

Input: Feature dataset

Output: Next day price prediction

Functionality:

- Train Linear Regression model
- Split data into training and testing sets
- Evaluate model using MAE and R^2 score
- Predict next day closing price
- Plot predicted vs actual values

Module 5: Visualization Dashboard

Input: Processed data

Output: Interactive charts

Functionality:

- Price trend chart
- RSI chart
- MACD chart
- Prediction vs Actual chart
- Display metrics such as latest close price and volume

3.2 Input/Output Structure

Input Flow:

Stock Ticker -> Download Data -> Feature Engineering -> Risk Analysis -> ML Prediction -> Visualization

User Input:

- Stock ticker symbol
- Start date
- End date

System Output:

- Technical indicators (RSI, MACD, Moving Averages)
- Risk metrics
- Interactive charts
- Prediction vs Actual graph
- Next day predicted closing price
- Downloadable CSV file

3.3 User Interaction Workflow

1. User enters stock ticker and date range in the sidebar.

2. System downloads stock data from Yahoo Finance.

3. Technical indicators and features are calculated.

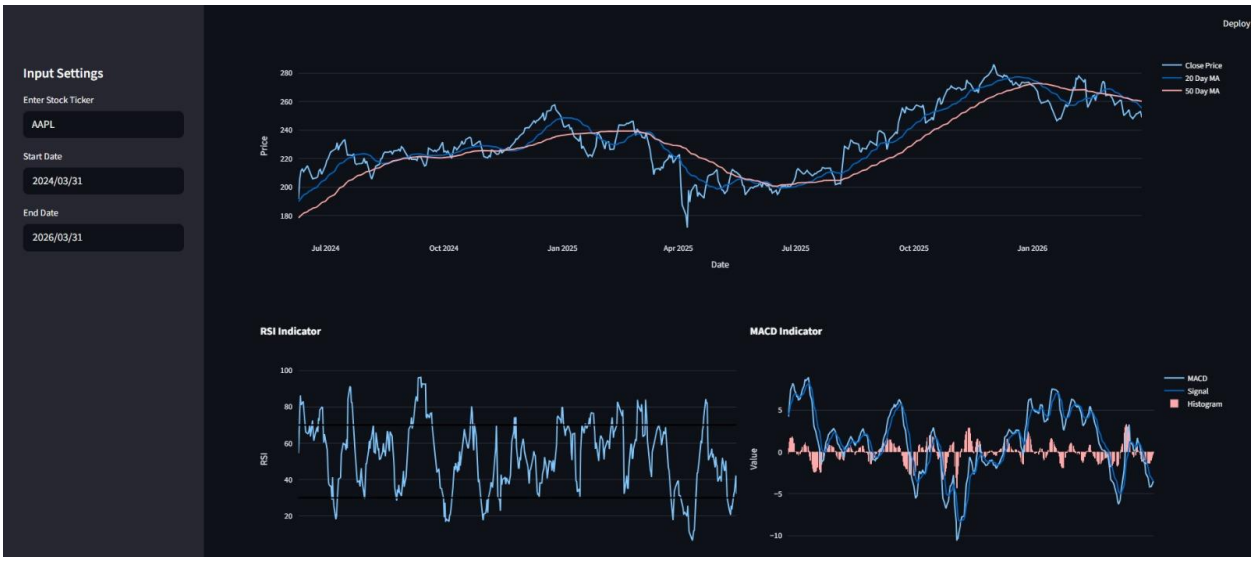
4. Risk metrics are computed.

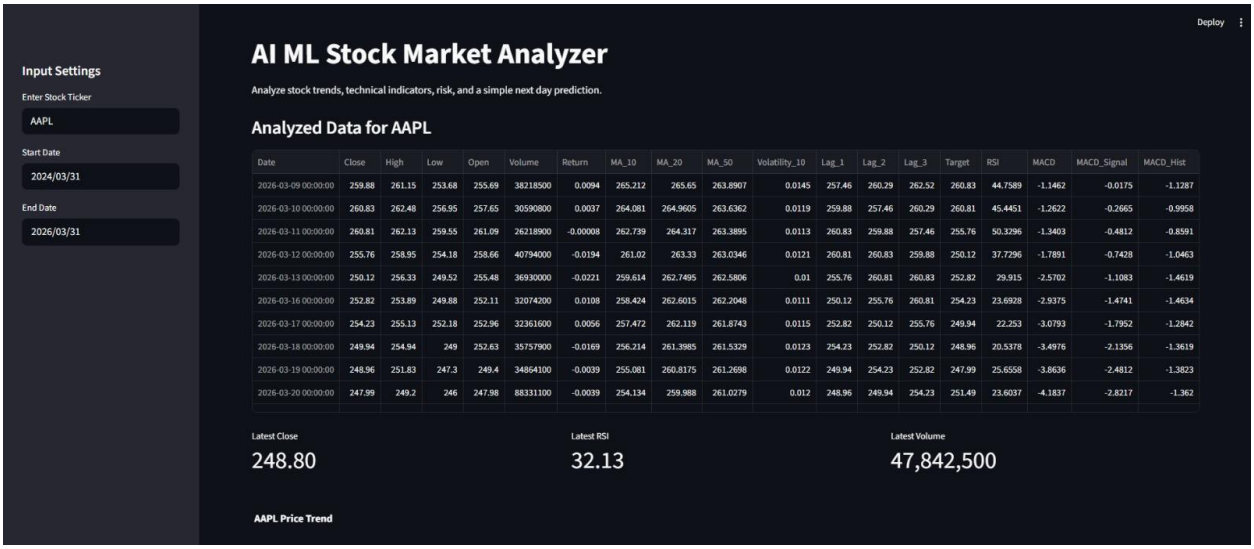
5. Machine learning model is trained.

6. Prediction results and evaluation metrics are displayed.

7. Charts and tables are shown.

8. User can download analyzed data as CSV.





Deploy

Input Settings

Enter Stock Ticker

AAPL

Start Date

2024/03/31

End Date

2026/03/31

AI ML Stock Market Analyzer

Analyze stock trends, technical indicators, risk, and a simple next day prediction.

Analyzed Data for AAPL

Date	Close	High	Low	Open	Volume	Return	MA_10	MA_20	MA_50	Volatility_10	Lag_1	Lag_2	Lag_3	Target	RSI	MACD	MACD_Signal	MACD_Hist
2026-03-09 00:00:00	259.88	261.15	253.68	255.69	38218500	0.0094	265.212	265.65	263.8907	0.0145	257.46	260.29	262.52	260.83	44.7589	-1.1462	-0.0175	-1.1287
2026-03-10 00:00:00	260.83	262.48	256.95	257.65	30590800	0.0037	264.081	264.9605	263.6362	0.0119	259.88	257.46	260.29	260.81	45.4451	-1.2622	-0.2665	-0.9958
2026-03-11 00:00:00	260.81	262.13	259.55	261.09	26218900	-0.00008	262.739	264.317	263.3895	0.0113	260.83	259.88	257.46	255.76	50.3296	-1.3403	-0.4812	-0.8591
2026-03-12 00:00:00	255.76	258.95	254.18	258.66	40794000	-0.0194	261.02	263.33	263.0346	0.0121	260.81	260.83	259.88	250.12	37.7296	-1.7891	-0.7428	-1.0463
2026-03-13 00:00:00	250.12	256.33	249.52	255.48	36930000	-0.0221	259.614	262.7495	262.5806	0.01	255.76	260.81	260.83	252.82	29.915	-2.5702	-1.1083	-1.4619
2026-03-16 00:00:00	252.82	253.89	249.88	252.11	32074200	0.0108	258.424	262.6015	262.2048	0.0111	250.12	255.76	260.81	254.23	23.6928	-2.9375	-1.4741	-1.4634
2026-03-17 00:00:00	254.23	255.13	252.18	252.96	32361600	0.0056	257.472	262.119	261.8743	0.0115	252.82	250.12	255.76	249.94	22.253	-3.0793	-1.7952	-1.2842
2026-03-18 00:00:00	249.94	254.94	249	252.63	35757900	-0.0169	256.214	261.3985	261.5329	0.0123	254.23	252.82	250.12	248.96	20.5378	-3.4976	-2.1356	-1.3619
2026-03-19 00:00:00	248.96	251.83	247.3	249.4	34864100	-0.0039	255.081	260.8175	261.2698	0.0122	249.94	254.23	252.82	247.99	25.6558	-3.8636	-2.4812	-1.3823
2026-03-20 00:00:00	247.99	249.2	246	247.98	88331100	-0.0039	254.134	259.988	261.0279	0.012	248.96	249.94	254.23	251.49	23.6037	-4.1837	-2.8217	-1.362

Latest Close

248.80

Latest RSI

32.13

Latest Volume

47,842,500

AAPL Price Trend

4. Non-Functional Requirements

4.1 Performance

- Data download and processing should complete within a few seconds
- Model training should run on CPU without GPU requirement
- Charts should load quickly
- Prediction should be generated instantly after training

4.2 Usability

- Simple web interface built with Streamlit
- User only needs to enter stock ticker and date range
- Interactive charts for better understanding
- Download button for analyzed data

4.3 Reliability

- Handles invalid ticker symbols
- Handles missing data
- Prevents model training if data is insufficient
- Uses exception handling for errors

4.4 Scalability

- More indicators can be added later
- More machine learning models can be integrated
- Can be deployed to cloud platforms
- Can be expanded into a portfolio analysis tool

4.5 Maintainability

- Modular functions for RSI, MACD, Risk Metrics, and Model Training
- Easy to modify feature engineering

- Code structure supports future improvements

4.6 Security

- Only reads stock data from Yahoo Finance
- No sensitive user data stored
- File downloads are limited to CSV export
- Safe user input handling

4.7 Resource Efficiency

- Uses Linear Regression which is lightweight
- Uses rolling calculations for indicators
- Works efficiently on standard laptops
- No heavy deep learning models required

4.8 Error Handling Strategy

- Displays error if ticker symbol is invalid
- Displays warning if not enough data for training
- Handles API failures
- Uses try-except blocks for safe execution

5. Technical Architecture

5.1 Component Architecture

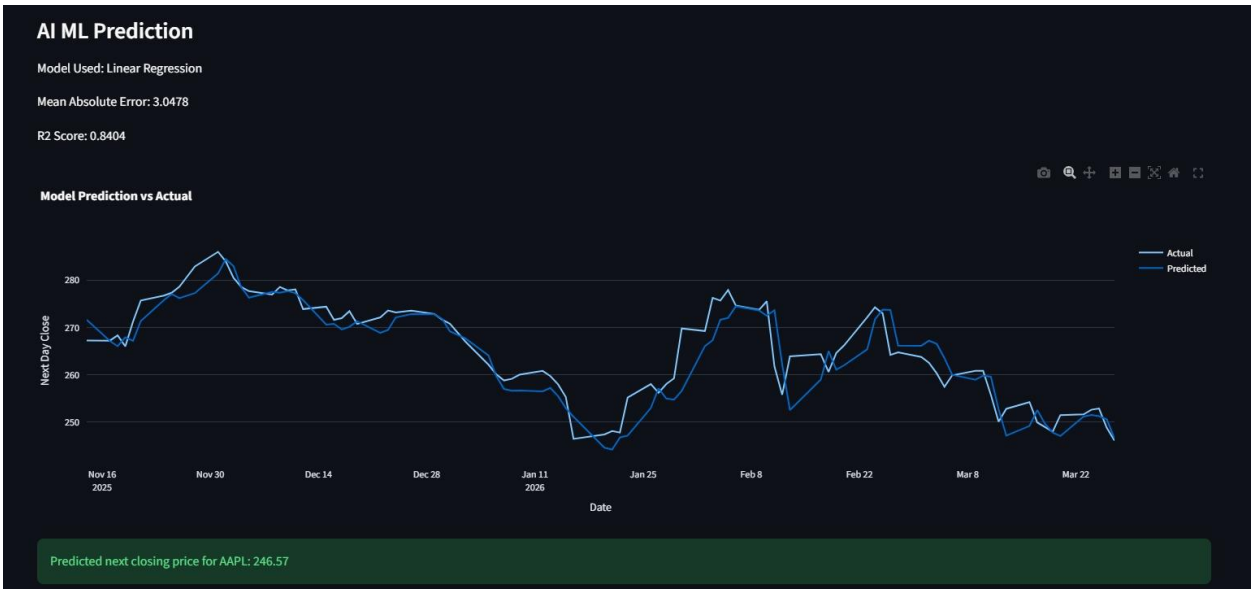
Core Components:

1. Data Loader Module – Downloads stock data
2. Feature Engineering Module – Creates indicators and lag features
3. Risk Analysis Module – Calculates volatility and Sharpe ratio
4. Machine Learning Module – Trains Linear Regression model
5. Visualization Module – Generates charts using Plotly
6. Streamlit UI Module – User interface and interaction

5.3 Technology Stack

- Programming Language: Python
- Web Framework: Streamlit
- Data Source: Yahoo Finance API
- Machine Learning: scikit-learn
- Data Processing: Pandas, NumPy

- Visualization: Plotly
- Model: Linear Regression



6. Implementation Details

6.1 Model Architecture

The system uses Linear Regression to predict the next day closing price based on technical indicators and lag features.

6.2 Data Preprocessing Pipeline

Steps:

- Download stock data and returns

- Calculate moving averages
- Calculate RSI and MACD
- Calculate volatility
- Create lag features
- Create target variable (next day closing price)
- Remove missing values

6.3 Training Configuration

- Model: Linear Regression
- Train-Test Split: 80% training, 20% testing
- Evaluation Metrics: MAE and R^2 Score
- Features Used:
    - Lag prices
    - Moving averages
    - Volatility
    - RSI
    - MACD

Risk Metrics

	Metric	Value
0	Average Daily Return	0.000743
1	Daily Volatility	0.018154
2	Expected Annual Return	0.187270
3	Annual Volatility	0.288190
4	Sharpe Ratio	0.649814

7. Dataset Description

7.1 Dataset Overview

- Data is downloaded from Yahoo Finance
- Contains historical stock prices
- Includes Open, High, Low, Close, Volume
- Date range selected by user
- Used for feature engineering and prediction

8. Model Selection Rationale

8.1 Linear Regression

- Simple and fast
- Works well for numerical time series features
- Easy to interpret
- Suitable for baseline prediction model

8.2 Feature-Based Prediction Strategy

The model uses:

- Moving averages for trend detection
- RSI for momentum analysis
- MACD for trend reversal signals
- Lag prices for time series prediction
- Volatility for risk measurement

These features help the model learn patterns in stock price movement.

9. Evaluation Methodology

9.1 Performance Metrics

- Mean Absolute Error (MAE)
- R^2 Score
- Prediction vs Actual graph

9.2 Validation Strategy

- Data split into training and testing sets

- Model trained on training data
- Predictions evaluated on testing data
- Performance metrics displayed in dashboard

10. Results and Findings

10.1 Evaluation Output

The system displays:

- Mean Absolute Error
- R^2 Score
- Prediction vs Actual chart
- Next day predicted closing price

10.2 Key Findings

- Technical indicators help improve prediction accuracy
- Lag features are important for time series prediction
- Linear Regression provides reasonable baseline predictions
- Risk metrics help understand stock volatility
- Visualization helps interpret stock trends easily

10.3 Observed System Behavior

- System downloads stock data successfully
- Indicators update automatically for selected date range
- Model trains and predicts next day price
- Charts display price trends and indicators
- User can download analyzed dataset

10.4 Interpretation of Findings

The project demonstrates that machine learning can be used to analyze stock market data and predict future prices. Although Linear Regression is a simple model, combining technical indicators and lag features allows the system to identify patterns in stock price movements. The project successfully integrates data analysis, visualization, and machine learning into a single interactive application.

11. Deployment and Usage

11.1 Installation Requirements

pip install streamlit yfinance pandas numpy plotly scikit-learn

11.2 Usage Instructions

1. Run the application:
   streamlit run app.py
2. Enter stock ticker in sidebar
3. Select start and end date
4. View analysis, charts, and prediction
5. Download analyzed data if required

12. Future Enhancements

12.1 Technical Improvements

- Use advanced models like Random Forest, XGBoost, LSTM
- Add multiple stock comparison
- Add portfolio optimization
- Add real-time stock data
- Improve prediction accuracy

12.2 Feature Additions

- Buy/Sell signal generator

- News sentiment analysis
- Portfolio risk analysis
- Automated trading signals
- Mobile dashboard

13. Conclusion

This project successfully implements an AI and Machine Learning based Stock Market Analyzer that integrates stock data collection, technical indicator calculation, risk analysis, machine learning prediction, and interactive visualization into a single system. The Streamlit dashboard allows users to analyze stock trends and predict future prices easily. The project demonstrates the practical application of data science, financial analysis, and machine learning in stock market prediction and decision support systems.